

TECHNICAL PORTFOLIO

Vo Hong Quan

AI Engineer: Reinforcement Learning for Robot Control

Learning-based control for robots: modeled, trained and benchmarked in simulation.

EMAIL

vohongquan.6524@gmail.com

PHONE

(+84) 363 645 485

GITHUB

github.com/vohongquann

WEBSITE

vohongquann.github.io/Portfolio

Professional Profile

Professional Summary

Robotics and AI engineer who builds learning-based controllers for robots. Works across the whole loop: modeling the physical system, training reinforcement learning policies, and measuring them against classical PID and LQR controllers so every claim rests on a number. Works mainly in Python, PyTorch and NVIDIA Isaac Lab. Looking for an AI or robotics engineering intern role where learned control meets real machines.

Core Competencies

REINFORCEMENT LEARNING PPO, SAC, reward design, hyperparameter tuning, frozen policy cascades	CONTROL SYSTEMS Cascaded PID, LQR, motor and sensor modeling, step-response analysis
ROBOTICS & SIMULATION NVIDIA Isaac Sim and Isaac Lab, PhysX, URDF from CAD, ROS 2	EMBEDDED DEPLOYMENT Policy deployed on ESP32; networks sized for an STM32 flight controller
MACHINE LEARNING & VISION PyTorch, TensorFlow, Keras, scikit-learn, OpenCV, YOLO	TESTING & EVALUATION Randomized evaluation episodes, baselines, pytest, reproducible scripts

Tools & Technologies

- Design / simulation: NVIDIA Isaac Sim, Isaac Lab, PhysX, MATLAB (Simulink, Simscape, RL Toolbox), URDF
- Programming / firmware: Python, C/C++, MATLAB
- Hardware / platforms: modeled Crazyflie 2.1 Brushless, Pololu Balboa 32U4, Quanser QUBE-Servo 2; ESP32
- Management / collaboration: Git, GitHub, Conda

Experience, Education & Achievements

Experience / Internship / Research

Feb 2025 – Present	Research Assistant - Robotics Lab, Ho Chi Minh City University of Technology and Engineering • Support reinforcement learning training and real-time testing of robot control systems. • Assist in designing reward functions and tuning hyperparameters for reinforcement learning agents.
--------------------	--

Education

DEGREE Bachelor of Robotics and Artificial Intelligence	INSTITUTION Ho Chi Minh City University of Technology and Engineering
TIME PERIOD Aug 2022 – Dec 2026 (expected)	HIGHLIGHTS GPA 3.22/4.0 Coursework: Machine Vision, Artificial Intelligence, Artificial Neural Networks, Practice of Artificial Intelligence

Certifications

- TOEIC Listening & Reading: 715/990 - IIG Vietnam - 2025

Selected Project Portfolio

01

Crazyflie-IsaacLab: RL-Tuned PID Gains for a Crazyflie Quadrotor

Small networks tune the PID gains of each layer of a four-layer flight cascade.

Individual project • Python, PyTorch, PPO, Isaac Lab • Jan 2026 – Sep 2026

Key result: 23–62% lower tracking error than the hand-tuned PID on all four layers.

02

BalancingRobot-IsaacLab: Self-Balancing Robot with an RL Cascade

A velocity policy turns speed and turn-rate commands into wheel torques; a position policy on top drives a Pololu Balboa to a goal, from raw IMU and encoders.

Individual project • Python, PPO, ROS 2, LQR, PID • Jun 2025 – Dec 2025

Key result: no falls in 768 randomized episodes; goals reached within 8 mm (median).

03

RotaryPendulum-IsaacLab: Learned Cascade PID for a Rotary Inverted Pendulum

A PPO policy writes the six gains of a two-loop PID that swings up and balances a Quanser QUBE-Servo 2 in simulation.

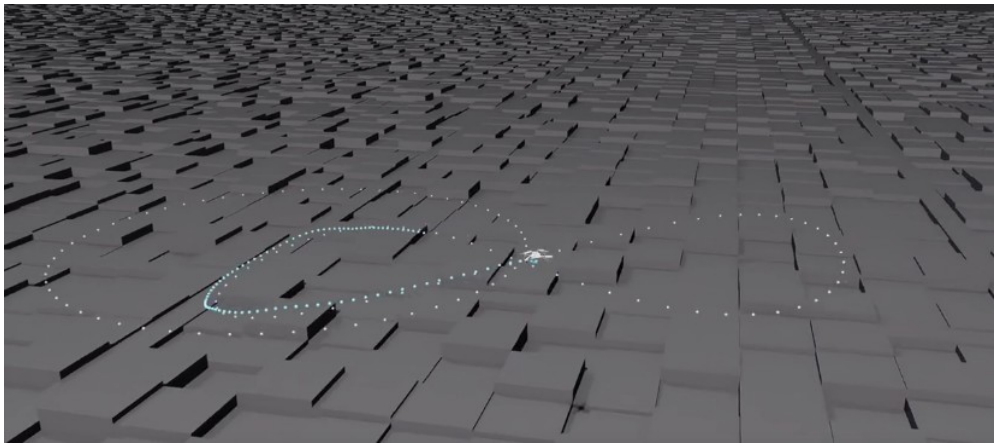
Individual project • Python, PPO, Isaac Lab, PhysX • Feb 2025 – May 2025

Key result: swing-up in 0.40 s across 256 robots with individual motor constants.

Crazyflie-IsaacLab: RL-Tuned PID Gains for a Crazyflie Quadrotor

A Crazyflie 2.1 Brushless flown in Isaac Lab 3.0 by a four-layer cascade (position, velocity, attitude, rate) whose PID gains are written, step by step, by small PPO networks.

ROLE Individual project: modeling, training, evaluation, documentation	TIME PERIOD 01/2026 – 09/2026
DOMAIN Robotics / Control / Reinforcement learning	LINKS GitHub: github.com/vohongquann/Crazyflie-IsaacLab



ENGINEERING PROBLEM Track position commands with a Crazyflie 2.1 Brushless through position 50 Hz, velocity 100 Hz, attitude 250 Hz and rate 500 Hz loops on a 1 kHz physics step, with networks small enough to run on the drone's STM32 microcontroller.	MY CONTRIBUTION - Modeled the drone: every constant in one file, each traced to the datasheet, system-identification papers or Bitcraze firmware. - Trained one MLP per layer with PPO (rsl_rl), bottom-up, freezing each before training the next. - Compared PID and RL gains layer by layer, wrote the report scripts, a pytest suite and a demo video.
--	---

Solution & System Architecture

Commands: random targets, circle, figure 8 (position layer)
Gain network (small MLP, PPO): observations → 9 PID gains of its layer, every step
Cascade: position 50 Hz → velocity 100 Hz → attitude 250 Hz → rate 500 Hz → motors
Crazyflie 2.1 Brushless model: motor curve, mixer, 1 kHz PhysX step (Isaac Lab 3.0)
Tracking error and motor chatter per layer: 64 drones, one episode

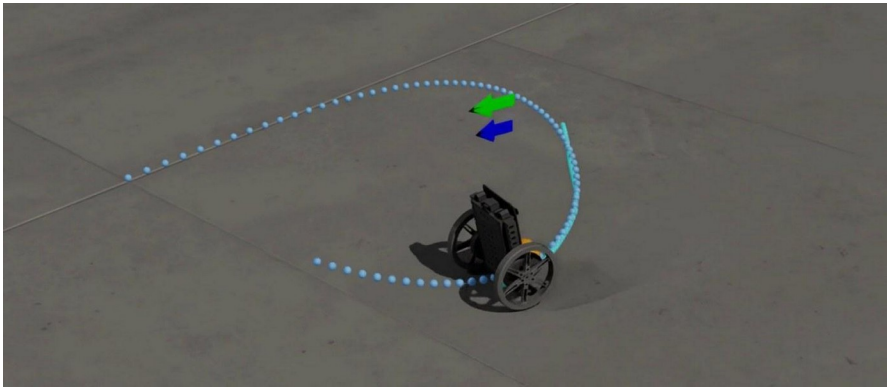
Measured Results

15.5° → 5.9° attitude error, PID → RL gains	0.85 → 0.46 m/s velocity error, PID → RL gains	2.6–6.3× motor chatter vs PID on the outer layers
---	--	---

BalancingRobot-IsaacLab: Self-Balancing Robot with an RL Cascade

A Pololu Balboa 32U4 that balances, follows a speed and turn-rate command and drives to a goal using only its IMU and wheel encoders, trained in Isaac Lab as a velocity policy with a position policy on top.

ROLE Individual project: modeling, training, evaluation, ROS 2 bridge	TIME PERIOD 06/2025 – 12/2025
DOMAIN Robotics / Control / Reinforcement learning	LINKS GitHub: github.com/vohongquann/BalancingRobot-IsaacLab



Frame from the demo video: the position cascade follows a figure 8 (green arrow: speed, blue: speed target).

ENGINEERING PROBLEM Balance an unstable two-wheeled robot, then drive it at a commanded speed and turn rate and to a goal pose, from a noisy accelerometer and gyro and wheel encoders, under random start tilt and pushes.	MY CONTRIBUTION - Modeled the robot: CAD to URDF, gearmotor and gearbox, IMU with datasheet noise and a complementary filter to pitch. - Trained a velocity policy (wheel torques), froze it, and trained a position policy on top (goal x, y, heading); each is frozen only after passing its evaluation. - Benchmarked against LQR and cascaded PID; built a ROS 2 bridge, a simulator-free test suite and training videos.
---	--

Solution & System Architecture

Raw accelerometer and gyro (datasheet noise), wheel encoders
Complementary filter → pitch; position policy (9 observations) → frozen velocity policy (8 observations)
Goal (x, y, heading) → speed and turn-rate command → common and turning wheel torque; LQR, PID and PID-gain policies compared
Pololu Balboa 32U4 model: 80 mm wheels, 50:1 gearmotors, LSM6DS33 IMU (Isaac Lab / PhysX)
768 evaluation episodes per policy: pushes, IMU noise, randomized mass, friction and centre of mass

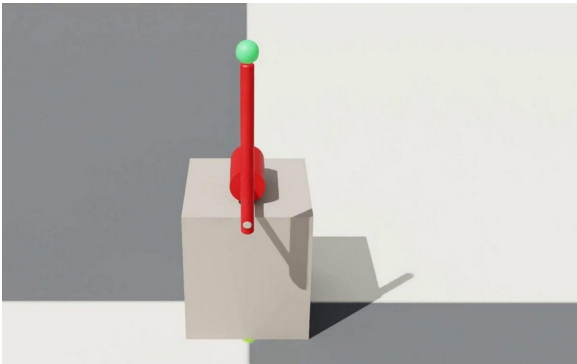
Measured Results

0 falls in 768 randomized episodes, every policy	8 mm median distance to the goal when the next goal comes	0.026 m/s speed-step error, settled in 0.36 s (LQR: 0.067 m/s, 1.1 s)
--	---	---

RotaryPendulum-IsaacLab: Learned Cascade PID for a Rotary Inverted Pendulum

A Quanser QUBE-Servo 2 rotary pendulum in Isaac Lab whose swing-up, balance and arm angle are held by a two-loop PID that a PPO policy retunes at 100 Hz.

ROLE Individual project: modeling, training, evaluation, documentation	TIME PERIOD 02/2025 – 05/2025
DOMAIN Robotics / Control / Reinforcement learning	LINKS GitHub: github.com/vohongquann/RotaryPendulum-IsaacLab



Frame from the demo video: pendulum balanced upright while the arm holds its commanded angle.

ENGINEERING PROBLEM Swing a rotary inverted pendulum up from hanging, balance it, and make the arm follow angle commands, with the QUBE-Servo 2's ± 12 V motor, quantized encoders and different motor constants on every simulated robot.	MY CONTRIBUTION - Modeled the QUBE-Servo 2: URDF built from CAD, motor chain PWM $\rightarrow$ voltage $\rightarrow$ current (back-EMF) $\rightarrow$ torque, quantized encoders. - Trained a PPO policy that writes the six gains of both PID loops. - Evaluated on 256 robots with a fixed scenario; published weights, evaluation, demo and training scripts, and sim-to-real notes.
--	---

Solution & System Architecture

Arm angle command; encoder angles and filtered velocities (12 observations)
PPO policy at 100 Hz: 12 observations $\rightarrow$ 6 PID gains (K_p , K_i , K_d of both loops)
Outer PID 250 Hz: arm error $\rightarrow$ pendulum setpoint; inner PID 500 Hz: pendulum error $\rightarrow$ voltage
QUBE-Servo 2 model: ± 12 V PWM, current, back-EMF, torque; PhysX at 500 Hz
256 simulated robots, each with its own motor constants: median and 5–95% band

Measured Results

0.40 s median swing-up time (95th percentile 0.48 s)	0.23–0.50 s median arm-step settling time	$\leq 1.4^\circ$ median overshoot, steady error $\leq 0.24^\circ$
--	---	--

Technical Evidence & Contact

Source code, results and videos for every project.

Repositories / Documents

01	Crazyflie-IsaacLab	Cascaded RL gain tuning, Crazyflie 2.1 Brushless: github.com/vohongquann/Crazyflie-IsaacLab
02	BalancingRobot-IsaacLab	Frozen RL cascade for a Pololu Balboa: github.com/vohongquann/BalancingRobot-IsaacLab
03	RotaryPendulum-IsaacLab	Learned cascade PID, QUBE-Servo 2: github.com/vohongquann/RotaryPendulum-IsaacLab
04	GitHub profile	All repositories: github.com/vohongquann
05	Portfolio website	Interactive demos and CV download: vohongquann.github.io/Portfolio



GitHub profile: github.com/vohongquann

Contact

EMAIL vohongquan.6524@gmail.com	PHONE (+84) 363 645 485
GITHUB github.com/vohongquann	WEBSITE vohongquann.github.io/Portfolio